

Synthèse des optimisations de performance

Simulation de colonie de fourmis — module de calcul backend

Contexte matériel

Processeur	Apple M4 Pro — 12 cœurs
Mémoire	24 Go
Système	macOS
Runtime	Go 1.26.4

Diagnostic

Avant toute mesure, le simple usage du programme suffisait à repérer le problème :

- Le module principal était perceptiblement lent à l'exécution, là où le calcul demandé par tick est trivial.
- Un seul cœur travaillait, visible dans le moniteur d'activité, alors que la machine en propose douze.
- Sur une exécution longue, la mémoire du processus grossissait à vue d'œil, sans jamais se stabiliser.

Résultats — les chiffres

Optimisation n°1 — Adressage de la grille

Module	Temps — avant	Temps — après	Opérations mémoire — avant	Opérations mémoire — après
Module principal	4,10 s	39,3 ms	66,3 millions	165 300
Module « recherche de nourriture »	3,85 s	42,3 ms	67,1 millions	165 100

Vérification indépendante (compteurs matériels du processeur, module principal) :

Compteur matériel	Avant	Après	Rapport
Instructions CPU exécutées	93,06 milliards	1,28 milliard	×72,5
Cycles d'horloge consommés	18,47 milliards	0,19 milliard	×96,0
Mémoire occupée au pic	19,4 Mo	10,6 Mo	×1,8

Optimisation n°2 — Parallélisation multi-cœurs

Métrique (scénario <code>large</code> , 256×256)	Avant (1 cœur)	Après (12 cœurs)
Temps d'exécution	134 ms	87 ms

Optimisation n°3 — Résolution d'une fuite de mémoire

Sur 60 000 cycles (`large`) : 26 Mo → **4,4 Go** en moins de 19 s avant correction ; stable à 8 Mo après.

Métrique (scénario <code>large</code> , 300 cycles)	Avant	Après	Rapport
Nombre d'allocations mémoire	1 506 703	19	≈79 300×
Mémoire allouée au total	26,1 Mo	16 Ko	≈1 624×

Vue d'ensemble cumulative — module principal, les 4 étapes, même scénario

Étape	Stratégie	Temps d'exécution	Mémoire allouée	Allocations	Δ vs étape précédente
<code>naive</code>	Référence : <code>clé texte (fmt.Sprintf) + map[string]uint32</code> , un seul cœur	12,25 s	1,52 Go	200 333 979	—
<code>flatgrid</code>	Suppression de la clé texte et de la <code>map</code> → tableau indexé	130,6 ms	26,1 Mo	1 506 705	×93,8 temps, ×133 allocations
<code>parallel</code>	+ worker pool sur 12 cœurs	87,1 ms	26,1 Mo (inchangée)	1 506 703 (inchangées)	×1,50 temps
<code>nogc</code>	+ suppression de l'historique inutile (<code>Ant.Trail</code>)	50,6 ms	16 Ko	19	×1,72 temps, ×79 300 allocations
Total cumulé	<code>naive</code> → <code>nogc</code>	×242 plus rapide	×95 000 moins de mémoire allouée	×10,5 million de fois moins d'allocations	—

Analyse — cause racine et résolution

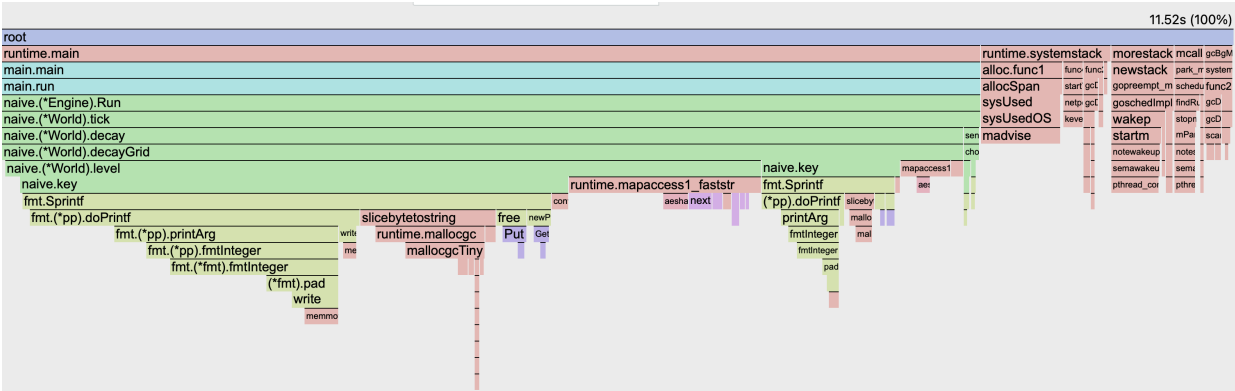
Optimisation n°1 — Adressage de la grille

Symptôme : le module principal met 4,10 s pour un scénario `medium` qui ne devrait prendre que quelques millisecondes — le temps explose avec la taille de la grille alors que la logique de simulation par tick est triviale. Même symptôme sur le module « recherche de nourriture », construit après coup et jamais corrigé.

Cause racine : `pprof` (profileur CPU intégré à Go, échantillonnage en cours d'exécution réelle) chiffre le coût. La grille (murs, nourriture, traces chimiques) est stockée dans des `map` : chaque lecture de case construit d'abord une clé texte (`fmt.Sprintf` , ex. `"42,17"`) avant la recherche. **76 % du temps CPU total** y passe — ligne par ligne (`pprof -list`) : 55,6 % pour la construction de clé (`naive.go:105`), 20,8 % pour la recherche `map` .

76 %

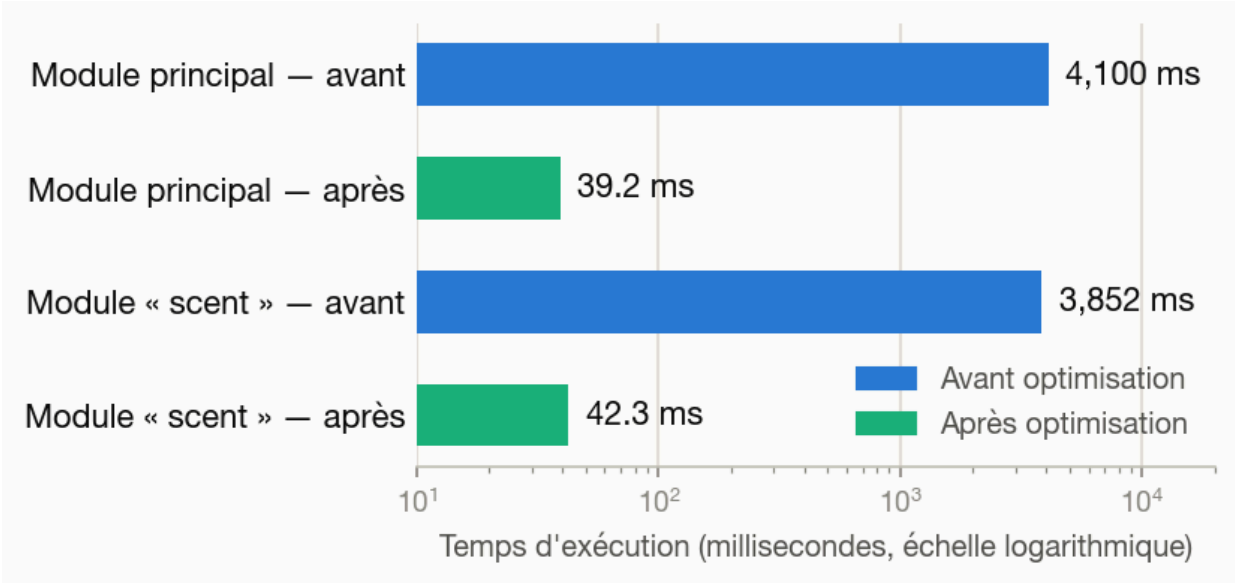
du temps CPU total consommé par la construction de clé texte et la recherche `map` .

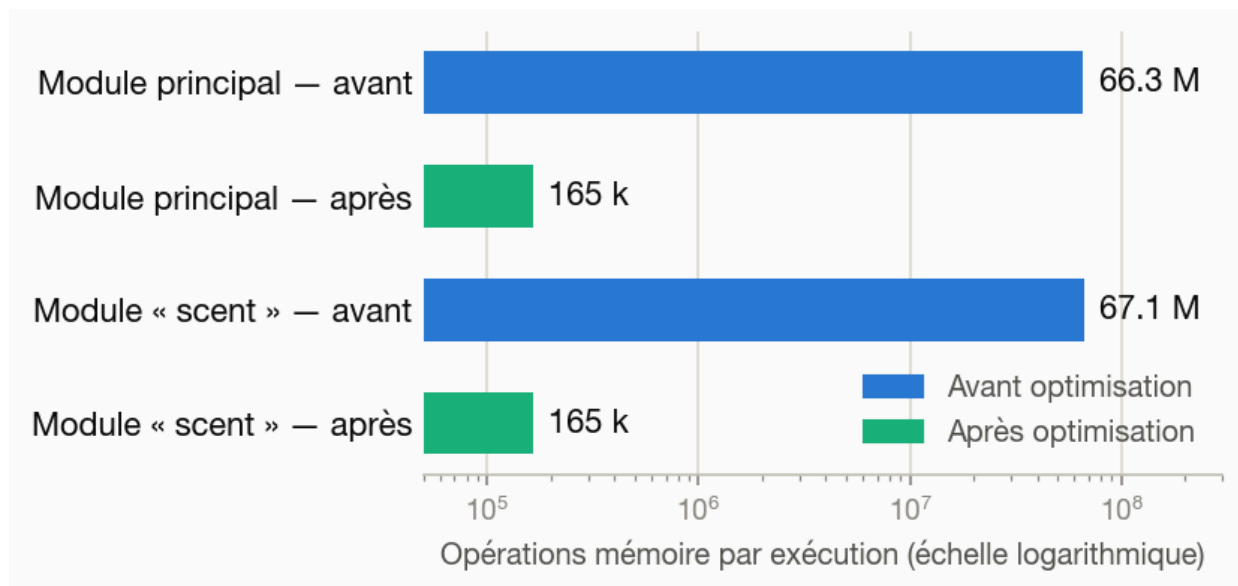


Confirmation par chronométrage isolé (`benchstat`) : lire une case par `map` coûte 52,96 ns, par index direct 0,313 ns — **169x plus lent**, hors de toute simulation. Trois mesures indépendantes (pourcentage `pprof` , arbre d'appel, chronométrage isolé) pointent la même cause.

Résolution : suppression de la clé texte et remplacement de chaque `map` de la grille par un tableau adressé par calcul d'index direct, sur les deux modules. Aucune autre logique changée — même ordre de traitement, mêmes règles, même hasard contrôlé. La vérification par compteurs matériels (`time -l`), indépendante de `pprof` , révèle en plus qu'avant correction un second cœur travaillait en tâche de fond (nettoyage mémoire automatique) : le temps CPU consommé (4,25 s) dépassait le temps réel écoulé (3,94 s).

Conclusion : opérations mémoire divisées par plus de 400 sur les deux modules ; $\times 93,8$ sur le temps d'exécution du module principal (détail chiffré en Résultats).





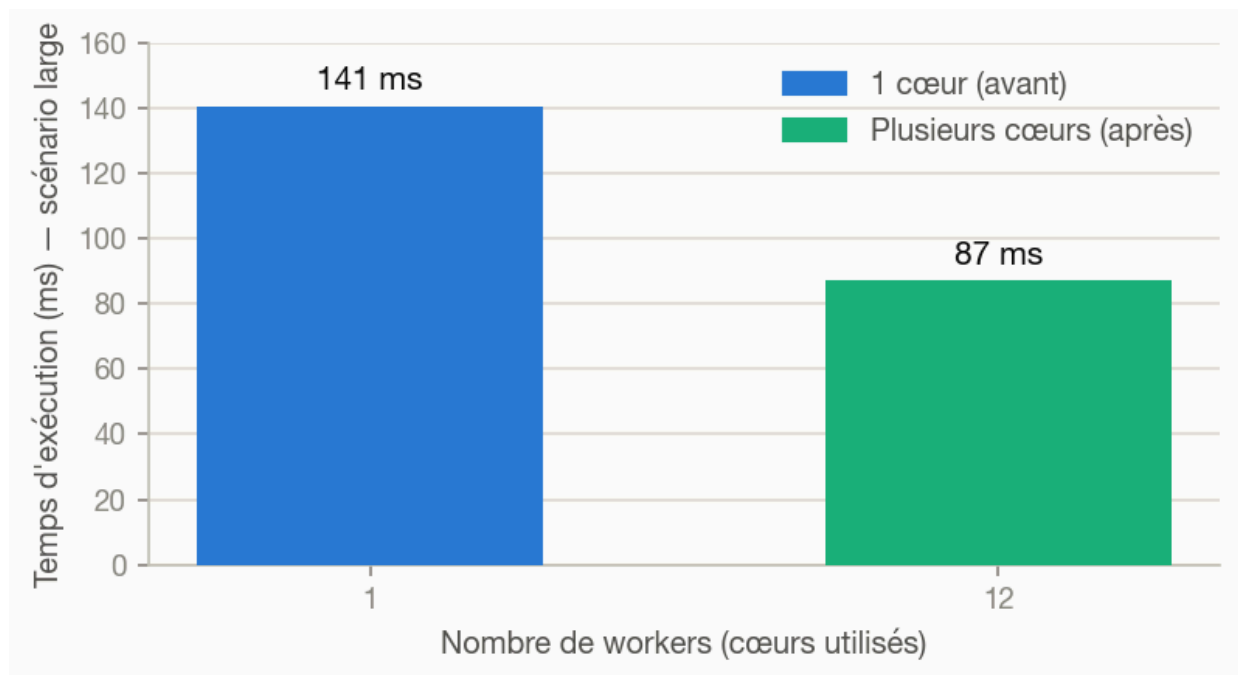
Optimisation n°2 — Parallélisation multi-cœurs

Symptôme : après la correction n°1, le programme reste mono-cœur sur une machine qui en propose douze.

Cause racine : aucun découpage du travail entre cœurs. Une partie du tick est parallélisable (réflexion des fourmis, évolution des traces chimiques) ; une autre doit rester série — le dépôt effectif sur le plateau (ramassage, retour au nid, marquage chimique), fourni par fourmi, pour trancher les conflits. C'est le **chemin critique** : la partie non parallélisable borne le gain, quel que soit le nombre de cœurs ajoutés.

Résolution : un groupe fixe de *workers* (jusqu'à 12) démarre une fois au lancement et se réutilise à chaque cycle ; le travail parallélisable leur est réparti, le dépôt reste série.

Conclusion : douze cœurs ne donnent pas douze fois plus vite, seulement $\times 1,6$ — au-delà d'un certain nombre de cœurs, c'est le chemin critique, pas le calcul, qui borne le temps.



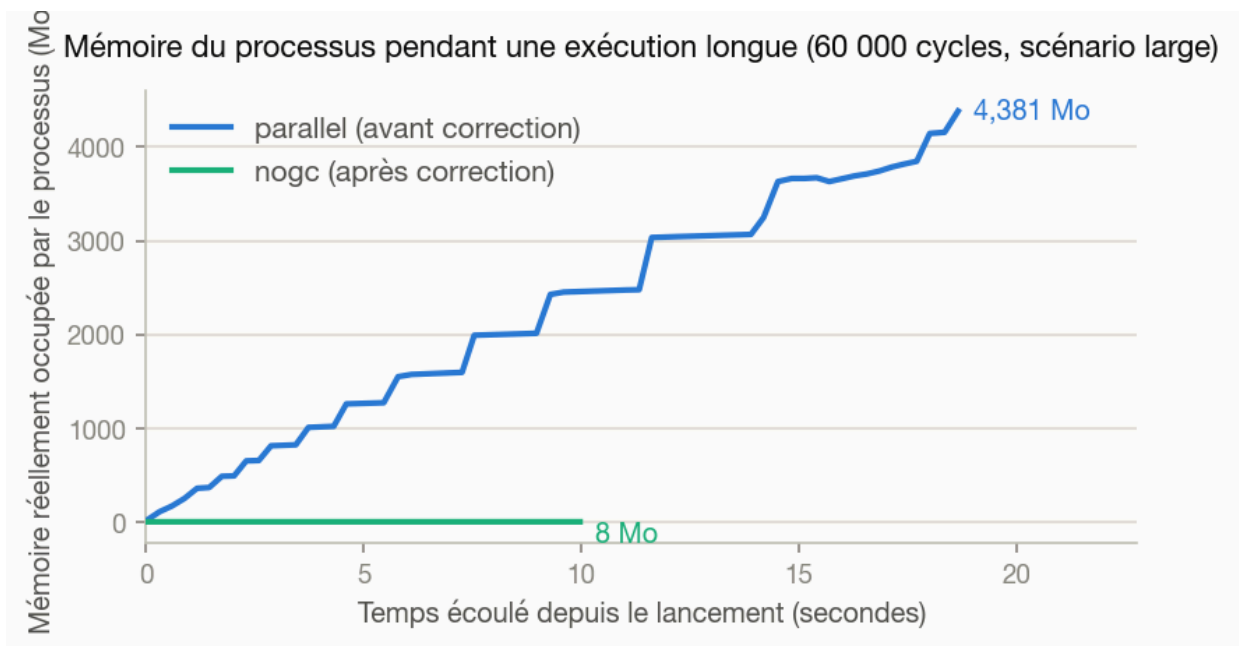
Optimisation n°3 — Résolution d'une fuite de mémoire

Symptôme : sur une exécution longue, la mémoire occupée augmente en continu sans jamais se stabiliser — 26 Mo à **4,4 Go** en moins de 19 s (60 000 cycles, scénario large).

Cause racine : chaque fourmi conserve l'historique complet des cases visitées depuis le début, dans une liste qui ne fait que grandir — ajoutée à une étape antérieure du projet, jamais relue, jamais nettoyée.

Résolution : suppression de cette liste. Rien d'autre modifié.

Conclusion : mémoire stable à 8 Mo du début à la fin. Effet secondaire favorable : cette liste alimentait aussi le chemin critique de l'optimisation n°2, qui devient plus court — 50,6 ms contre 87 ms précédemment, $\times 1,7$ supplémentaire, $\times 2,7$ depuis le tout premier module.



Méthode

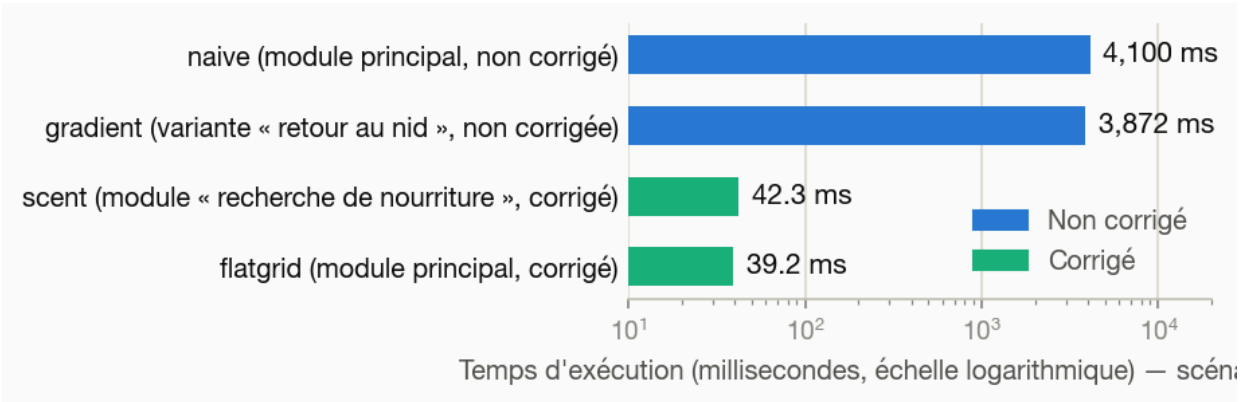
Outil	Rôle	Ce qu'il a montré ici
<code>pprof</code>	Profilage CPU/mémoire réel, ligne de code exacte.	Cible à corriger : <code>naive.key</code> + la <code>map</code> , 76 % du CPU (Analyse).
<code>go test -bench + benchstat</code>	Micro-mesures moyennées sur ≥6 exécutions.	Chiffrage avant/après par module (Résultats, Annexe).
<code>hyperfine</code>	Temps du binaire complet, 10 exécutions, chauffe incluse.	Confirme l'ordre de grandeur de <code>benchstat</code> par une mesure indépendante.
<code>time -l</code> (compteurs matériels)	Instructions/cycles CPU, mémoire réelle — indépendant de l'outillage Go.	Confirme le gain une 3 ^e fois ; a révélé un second cœur actif en tâche de fond avant correction (n°1).
Tests de non-régression (<code>go test</code>)	Comparaison bit à bit du résultat produit, avant/après.	Les quatre corrections passent ; <code>-race</code> en plus pour n°2/n°3 : aucune donnée partagée entre workers.
Échantillonnage RSS (<code>ps</code> , 1x/s)	Mémoire réellement occupée pendant l'exécution, seconde par seconde.	Graphique de fuite mémoire (n°3) : croissance continue avant, ligne plate après.

Interprétation des résultats — conclusion

Observation : le tableau cumulatif de Résultats affiche ×242 sur le temps total, mais cette valeur unique cache trois contributions très inégales (×93,8, ×1,50, ×1,72) — la lecture qui suit explique pourquoi cet écart n'est pas un hasard.

Constat	Lecture
Le gain vient d'arrêter de gâcher des ressources, pas d'en ajouter.	La correction n°1 (algorithmique, zéro ressource ajoutée) fait ×93,8 à elle seule. Les corrections n°2 et n°3, qui ajoutent des cœurs puis suppriment un gaspillage mémoire, n'apportent « que » ×1,5 et ×1,7 — et seulement une fois n°1 en place. Paralléliser la version d'origine telle quelle n'a pas été testé, mais son profil ne suggère rien de mieux : le même goulot (la <code>map</code>) domine quel que soit le nombre de cœurs.
Le gain se multiplie, il ne s'additionne pas.	$\times 242 = 93,8 \times 1,5 \times 1,72$, pas leur somme. Chaque correction s'applique à un programme déjà plus rapide, donc son effet relatif grandit.
Les trois corrections n'ont pas la même nature.	Les deux premières se lisent sur un chronomètre. La troisième (fuite mémoire) a un gain de vitesse modeste (×1,7) ; sa vraie valeur est la stabilité — sans elle, une exécution longue épuise la mémoire disponible et provoque un arrêt, indépendamment de la vitesse. Seul le suivi dans le temps (graphique de l'optimisation n°3) la révèle.

Annexe



Temps d'exécution et mémoire — micro-mesures (go test -bench + benchstat , 6 répétitions)

Module	Scénario	Temps par exécution	Mémoire allouée	Opérations mémoire
naive	small	475,9 ms	41,5 Mo	8,27 M
naive	medium	4,10 s	397,8 Mo	66,3 M
gradient	small	473,8 ms	41,5 Mo	8,27 M
gradient	medium	3,87 s	397,4 Mo	66,3 M
flatgrid	small	5,4 ms	1,34 Mo	25 200
flatgrid	medium	39,3 ms	8,57 Mo	165 300
scent	small	5,6 ms	1,34 Mo	25 200
scent	medium	42,3 ms	8,57 Mo	165 100
parallel	medium	36,3 ms	8,58 Mo	165 300
nogc	medium	28,1 ms	592 Ko	440

Temps du programme complet, démarrage inclus (hyperfine , 10 répétitions, medium)

Module	Temps moyen	Écart-type	Facteur vs le plus rapide
flatgrid	42,2 ms	± 0,7 ms	×1,0 (référence)
scent	45,8 ms	± 0,5 ms	×1,08
gradient	4,007 s	± 0,148 s	×94,9
naive	4,214 s	± 0,020 s	×99,8

Disponibilité sous charge HTTP réelle (vegeta , harnais web antweb)

Environnement	Disponibilité	Débit effectif	Latence p50	Latence p99
Déployé (ant-go.leo-bessin.dev , internet + proxy)	100,00 % (0 erreur)	49,97 req/s	30,8 ms	74,8 ms
Local (localhost:8080 , direct, même charge)	100,00 % (0 erreur)	—	0,69 ms	1,16 ms

Même charge (vegeta , 50 req/s, 30 s, GET /), même code — seul l'environnement réseau change. En local, aucun trajet internet ni proxy : l'écart (x64 sur le p99) mesure donc le coût du réseau et du proxy de déploiement, pas celui de l'application.

Mesure indicative (un seul run, sans warmup) — protocole moins rigoureux que hyperfine / benchstat (voir Méthode). Porte sur le harnais web, pas sur le moteur de simulation mesuré par ailleurs dans ce document.